

1. Main Idea Of The Project

One-line idea

The project takes a large Materials Project dataset, filters it to lithium compounds, scores India-focused feasibility, compares classical ML with Qiskit-based QML, and finally ranks exact lithium compounds for decision support.

This project is not only a machine learning experiment. The final product is a DSS, which means Decision Support System. A DSS does not make a final business decision by itself. It gives a ranked list and evidence so a person can make a better decision.

The main output is a ranked list of exact compound formulas such as $\text{SrLi}_2\text{SiO}_4$, $\text{Li}_2\text{MnGeS}_4$, and LiGaSiO_4 . Battery families are shown only as background context. The recommendation is not limited to names like LFP or LMFP.

The safe story is: XGBoost is the strong present-day classical benchmark. Qiskit Statevector QML is the quantum-future experiment. The final DSS is quantum-led because QML gives the first stability signal and XGBoost acts as a corrective backup.

2. What We Accomplished

Project part	Verified current value
Raw Materials Project snapshot	210,579 rows from 3 parquet shards
Lithium India scored dataset	24,957 rows and 37 columns
Final India battery shortlist	629 rows and 21 columns
DSS compound ranking	629 ranked exact compounds
QML-ready dataset	1,000 balanced rows and 27 columns
Exhaustive QML tuning	8,388 saved configurations
Repeated-split validation	10 random train/test splits

3. What We Can Do With The Results

- Use the DSS ranking as a first shortlist of lithium compounds for research discussion.
- Compare compounds by stability probability, India feasibility score, and predicted energy above hull.
- Use XGBoost as a strong classical baseline to show what current practical ML can do.
- Use Qiskit Statevector QML as a student-level quantum feature-space experiment.

- Explain to judges that this is not a claim of real quantum hardware advantage. It is a simulator-based QML comparison inside a DSS.
- Send the top compounds for deeper chemistry review, lab validation, or official Materials Project checking before any business decision.

4. Important Words In Simple Language

Word	Meaning
DSS	Decision Support System. It helps humans decide by showing ranked options and evidence.
Material formula	The chemical formula of a compound, like LiFePO_4 or $\text{SrLi}_2\text{SiO}_4$.
material_id	The Materials Project ID for a compound. It is like a database ID.
energy above hull	A stability number. Lower is usually better. Zero means the material is predicted to be stable in the dataset.
band gap	A materials property related to electronic behavior.
XGBoost	A strong classical ML model that builds many decision trees and combines them.
Logistic Regression	A simple classical baseline classifier. It is easier to beat than XGBoost.
QML	Quantum Machine Learning. In this project it means a quantum-kernel classifier simulated with Qiskit ideas.
Qubit	A quantum bit. Here, each selected feature is encoded into one qubit.
Statevector	A full mathematical description of the quantum state in a simulator.
Quantum kernel	A similarity score between two materials after encoding them into a quantum feature space.
SVM C	A setting of the SVM classifier. Larger C usually tries harder to fit the training data.
Precision	When the model says stable, how often it is correct.
Recall	Out of all truly stable materials, how many the model found.
F1 score	A balance between precision and recall.

5. Math Formulas Used In The Project

The formulas below were checked by compiling a LaTeX file before this Word document was created. They are written in LaTeX form so teammates can paste them into notes or slides.

$\theta_i = \pi x_i$	Quantum angle encoding. A scaled feature value x_i is turned into a rotation angle.
$\phi_{i,j} = \pi x_i x_j$	Controlled phase angle. It uses two neighboring feature values to

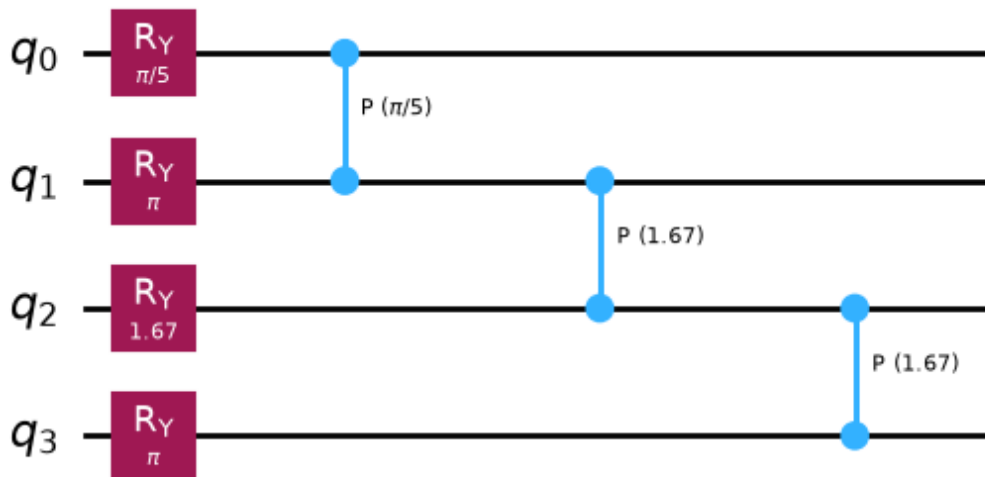
	create pairwise interaction.
$K(x, y) = \langle \phi(x) \phi(y) \rangle ^2$	Quantum kernel. It measures similarity between two encoded quantum states.
$2^n = 2^4 = 16$	Statevector size. A 4-qubit circuit has 16 amplitudes in the simulator.
$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$	Accuracy. Out of all predictions, how many were correct.
$Precision = \frac{TP}{TP + FP}$	Precision for stable class. Out of predicted stable materials, how many were truly stable.
$Recall = \frac{TP}{TP + FN}$	Recall for stable class. Out of truly stable materials, how many were found.
$F_1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$	F1 score. A single score balancing precision and recall.

6. Worked Math Example From The Qiskit Cell

The notebook uses this example feature vector in the Qiskit circuit cell:

$x = [0.20, 1.00, 0.53, 1.00]$		These are scaled feature values. They are already between 0 and 1.
Step	Calculation	Simple meaning
Step 1	$\theta_0 = \pi \times 0.20 = 0.6283$	The first feature becomes a rotation angle of about 0.6283 radians. This is close to $\pi/5$.
Step 2	$\theta_1 = \pi \times 1.00 = 3.1416$	The second feature becomes π radians.
Step 3	$\theta_2 = \pi \times 0.53 = 1.6650$	The third feature becomes about 1.665 radians.
Step 4	$\theta_3 = \pi \times 1.00 = 3.1416$	The fourth feature also becomes π radians.
Step 5	$\phi_{0,1} = \pi \times 0.20 \times 1.00 = 0.6283$	The first controlled phase uses feature 0 and feature 1.
Step 6	$\phi_{1,2} = \pi \times 1.00 \times 0.53 = 1.6650$	The second controlled phase uses feature 1 and feature 2.
Step 7	$\phi_{2,3} = \pi \times 0.53 \times 1.00 = 1.6650$	The third controlled phase uses feature 2 and feature 3.
Step 8	$2^4 = 16$	Because the circuit has 4 qubits, Qiskit Statevector stores 16 amplitudes.

The actual Qiskit matplotlib circuit saved in Main.ipynb is shown below:



7. Main Result Tables

Top 10 current DSS compound recommendations:

dss_rank	formula	material_id	hybrid_stable_probability	india_feasibility_score	predicted_energy_above_hull_clipped
1	SrLi2SiO4	mp-1191141	0.8401	87	0.0
2	Li2MnGeS4	mp-3268644	0.869	81	0.0009
3	LiGaSiO4	mp-18147	0.8329	87	0.0024
4	SrLi2SnS4	mp-3210567	0.8782	78	0.0
5	Rb2Li2SnS4	mp-3205208	0.8759	78	0.0
6	Cs2Li2SnS4	mp-3209528	0.8755	78	0.0
7	BaLi2SnS4	mp-3211122	0.876	78	0.0
8	Li2ZnGeS4	mp-1210804	0.8639	78	0.0
9	Rb2Li2TiS4	mp-3215312	0.8504	80	0.0
10	K2Li2TiS4	mp-3205916	0.8503	80	0.0

Main model metrics from current CSV outputs:

Model	Accuracy	Stable precision	Stable recall	Stable F1
QML quantum kernel	0.81	0.787	0.85	0.8173
Tuned QML quantum kernel	0.82	0.7963	0.86	0.8269

Improved QML separate section	0.82	0.8077	0.84	0.8235
Improved QML threshold tuning	0.83	0.8	0.88	0.8381
Improved QML kernel alignment	0.82	0.7857	0.88	0.8302
XGBoost same QML data	0.83	0.8367	0.82	0.8283
XGBoost full project	0.9091	0.73	0.7	0.71

8. How To Read The Notebook

Every cell below is explained in order. For code cells, the table maps each line number to what that line does. For markdown cells, the table explains what the written text is trying to communicate.

Important note for teammates

Do not memorize every line blindly. First understand the project flow: load data, show dataset filtering, rank materials, show Qiskit circuit, compare model results, then conclude safely.

Cell 1: Markdown

Purpose: Introduces the notebook title and the project goal.

First line: # Battery Materials DSS QML

Line	Notebook text	Meaning
1	# Battery Materials DSS QML	Main title of the notebook.
2	(blank line)	Blank line. It gives breathing space between text blocks.
3	**Presentation notebook**	Bold text used to highlight an important label or idea.
4	(blank line)	Blank line. It gives breathing space between text blocks.
5	Goal: build a student-level Decision Support System for lithium-ion battery	Explanation text written for the presentation audience.
6	material recommendation using Materials Project data, India-focused screening,	Explanation text written for the presentation audience.
7	XGBoost, and Qiskit Statevector QML comparison.	Explanation text written for the presentation audience.

Cell 2: Markdown

Purpose: Sets the honest student-level story: DSS first, quantum experiment second.

First line: ## Student-Level Project Flow

Line	Notebook text	Meaning
------	---------------	---------

1	## Student-Level Project Flow	Section heading. It tells the reader what the next part is about.
2	(blank line)	Blank line. It gives breathing space between text blocks.
3	This project is presented as a DSS first and a QML exploration second.	Explanation text written for the presentation audience.
4	(blank line)	Blank line. It gives breathing space between text blocks.
5	1. XGBoost is the strong current classical benchmark for structured materials	Numbered point in the project explanation.
6	data.	Explanation text written for the presentation audience.
7	2. Logistic Regression is the simple classical baseline.	Numbered point in the project explanation.
8	3. Qiskit Statevector QML is the quantum-future experiment.	Numbered point in the project explanation.
9	4. The final recommendation is made through DSS ranking tables, not through an	Numbered point in the project explanation.
10	unsupported claim of quantum advantage.	Explanation text written for the presentation audience.
11	(blank line)	Blank line. It gives breathing space between text blocks.
12	**Why quantum is included**	Bold text used to highlight an important label or idea.
13	(blank line)	Blank line. It gives breathing space between text blocks.
14	Battery materials are controlled by atomic and electronic behavior. That	Explanation text written for the presentation audience.
15	behavior is quantum mechanical. Classical ML is useful today, but future	Explanation text written for the presentation audience.
16	quantum models may represent material interactions more naturally. This project	Explanation text written for the presentation audience.
17	therefore uses Qiskit Statevector QML as a safe first step toward quantum-assisted	Explanation text written for the presentation audience.
18	materials discovery.	Explanation text written for the presentation audience.

Cell 3: Code

Purpose: Imports the tools and defines the CSV loading helper.

First line: from pathlib import Path

Line	Code	Simple explanation
1	from pathlib import Path	Imports Path, which helps the notebook handle file and folder paths cleanly.
2	import pandas as pd	Imports pandas. Pandas is used to read CSV files and work with tables.
3	import matplotlib.pyplot as plt	Imports matplotlib plotting tools so the notebook can draw charts.

4	<code>from IPython.display import display</code>	Imports display, which shows pandas tables clearly inside the notebook.
5	<code>from qiskit import QuantumCircuit</code>	Imports Qiskit QuantumCircuit. This is used to build the visible quantum circuit.
6	<code>from qiskit.quantum_info import Statevector</code>	Imports Qiskit Statevector. This simulates the quantum state created by the circuit.
7	<code>from sklearn.metrics import accuracy_score</code>	Imports accuracy_score from scikit-learn so the notebook can measure model performance.
8	<code>from sklearn.metrics import confusion_matrix</code>	Imports confusion_matrix from scikit-learn so the notebook can measure model performance.
9	<code>from sklearn.metrics import f1_score</code>	Imports f1_score from scikit-learn so the notebook can measure model performance.
10	<code>from sklearn.metrics import precision_score</code>	Imports precision_score from scikit-learn so the notebook can measure model performance.
11	<code>from sklearn.metrics import recall_score</code>	Imports recall_score from scikit-learn so the notebook can measure model performance.
12	(blank line)	Blank line. It separates blocks so the code is easier to read.
13	<code>project_folder = Path.cwd()</code>	Stores the current project folder. The notebook expects to be run from the project root.
14	<code>processed_folder = project_folder / "data" / "processed"</code>	Builds the path to the processed data folder, where the CSV outputs are stored.
15	(blank line)	Blank line. It separates blocks so the code is easier to read.
16	<code>def read_project_data(file_name):</code>	Defines a small helper function so each CSV file can be loaded with one simple line.
17	<code> return pd.read_csv(processed_folder / file_name)</code>	Reads a CSV file from the processed data folder and returns it as a pandas dataframe.

Cell 4: Code

Purpose: Shows how the dataset becomes smaller through the pipeline.

Important output: raw Materials Project rows = 210,579; lithium rows = 24,957; QML-ready rows = 1,000; final shortlist rows = 629.

First line: `lithium_scored_dataframe = read_project_data("lithium india scored.csv")`

Line	Code	Simple explanation
1	<code>lithium_scored_dataframe = read_project_data("lithium india scored.csv")</code>	Loads the file lithium india scored.csv into a dataframe.
2	<code>qml_ready_dataframe = read_project_data("qml_ready_lithium_india.csv")</code>	Loads the file qml_ready_lithium_india.csv into a dataframe.
3	<code>final_shortlist_dataframe = read_project_data("final india battery shortlist.csv")</code>	Loads the file final india battery shortlist.csv into a dataframe.
4	(blank line)	Blank line. It separates blocks so the code is easier to read.

5	<code>raw_data_folder = project_folder / "data" / "raw" / "materials_project_hf"</code>	Points to the folder where the raw Materials Project parquet files are stored.
6	<code>raw_materials_project_row_count = 0</code>	Starts the raw row counter at zero before adding rows from each parquet shard.
7	(blank line)	Blank line. It separates blocks so the code is easier to read.
8	<code>for raw_file_path in sorted(raw_data_folder.glob("*.parquet")):</code>	Loops through every raw parquet shard in sorted order.
9	<code> raw_file_dataframe = pd.read_parquet(raw_file_path)</code>	Reads one raw parquet shard into a dataframe.
10	<code> raw_materials_project_row_count = raw_materials_project_row_count + len(raw_file_dataframe)</code>	Adds the number of rows from the current raw shard to the running total.
11	(blank line)	Blank line. It separates blocks so the code is easier to read.
12	<code>pipeline_counts_dataframe = pd.DataFrame([</code>	Creates a small table manually from rows written in the notebook.
13	<code> {"stage": "Raw MP snapshot", "rows": raw_materials_project_row_count},</code>	Creates one row of a manually built table using column names and values.
14	<code> {"stage": "Lithium dataset", "rows": len(lithium_scored_dataframe)},</code>	Creates one row of a manually built table using column names and values.
15	<code> {"stage": "QML-ready dataset", "rows": len(qml_ready_dataframe)},</code>	Creates one row of a manually built table using column names and values.
16	<code> {"stage": "Final shortlist", "rows": len(final_shortlist_dataframe)},</code>	Creates one row of a manually built table using column names and values.
17	<code>])</code>	Closes the current list, dictionary, or function call.
18	(blank line)	Blank line. It separates blocks so the code is easier to read.
19	<code>plt.figure(figsize=(9, 5))</code>	Creates a new chart area with the chosen figure size.
20	<code>plt.bar(pipeline_counts_dataframe["stage"], pipeline_counts_dataframe["rows"])</code>	Draws a bar chart from the selected dataframe columns.
21	<code>plt.title("Dataset Size Through The Pipeline")</code>	Adds the chart title.
22	<code>plt.xlabel("Pipeline stage")</code>	Adds the x-axis label.
23	<code>plt.ylabel("Rows")</code>	Adds the y-axis label.
24	<code>plt.xticks(rotation=35)</code>	Rotates or labels the x-axis tick text so it is readable.
25	<code>plt.grid(axis="y", alpha=0.25)</code>	Adds a light grid to make chart values easier to compare.
26	<code>plt.show()</code>	Shows the chart output in the notebook.

Cell 5: Code

Purpose: Counts the most common battery families in the lithium dataset.

Important output: the chart/table shows which battery families appear most in the lithium dataset.

First line: `lithium_scored_dataframe = read_project_data("lithium india scored.csv")`

Line	Code	Simple explanation
1	<code>lithium_scored_dataframe = read_project_data("lithium india scored.csv")</code>	Loads the file lithium india scored.csv into a dataframe.
2	(blank line)	Blank line. It separates blocks so the code is easier to read.
3	<code>family_count_series = lithium_scored_dataframe["battery_family"].value_counts()</code>	Counts how many rows belong to each category.
4	<code>top_family_count_series = family_count_series.head(8)</code>	Keeps only the first few rows for a short display table.
5	<code>family_counts_dataframe = top_family_count_series.reset_index()</code>	Converts a counted Series back into a normal dataframe.
6	<code>family_counts_dataframe.columns = ["battery_family", "rows"]</code>	Renames the columns so the output table is easier to understand.
7	(blank line)	Blank line. It separates blocks so the code is easier to read.
8	<code>display(family_counts_dataframe)</code>	Displays the dataframe output in the notebook.
9	(blank line)	Blank line. It separates blocks so the code is easier to read.
10	<code>plt.figure(figsize=(9, 5))</code>	Creates a new chart area with the chosen figure size.
11	<code>plt.bar(family_counts_dataframe["battery_family"], family_counts_dataframe["rows"])</code>	Draws a bar chart from the selected dataframe columns.
12	<code>plt.title("Top Lithium Battery Families In Full Dataset")</code>	Adds the chart title.
13	<code>plt.xlabel("Battery family")</code>	Adds the x-axis label.
14	<code>plt.ylabel("Rows")</code>	Adds the y-axis label.
15	<code>plt.xticks(rotation=35)</code>	Rotates or labels the x-axis tick text so it is readable.
16	<code>plt.grid(axis="y", alpha=0.25)</code>	Adds a light grid to make chart values easier to compare.
17	<code>plt.show()</code>	Shows the chart output in the notebook.

Cell 6: Code

Purpose: Shows how many materials fall under each India feasibility decision label.

Important output: the chart/table shows how strict the India-focused screening is.

First line: `lithium_scored_dataframe = read_project_data("lithium india scored.csv")`

Line	Code	Simple explanation
1	<code>lithium_scored_dataframe = read_project_data("lithium india scored.csv")</code>	Loads the file lithium india scored.csv into a dataframe.
2	(blank line)	Blank line. It separates blocks so the code is easier to read.
3	<code>decision_count_series = lithium_scored_dataframe["india_decision_label"].value_counts()</code>	Counts how many rows belong to each category.
4	<code>decision_counts_dataframe =</code>	Converts a counted Series back into a normal dataframe.

	<code>decision_count_series.reset_index()</code>	
5	<code>decision_counts_dataframe.columns = ["india_decision_label", "rows"]</code>	Renames the columns so the output table is easier to understand.
6	(blank line)	Blank line. It separates blocks so the code is easier to read.
7	<code>display(decision_counts_dataframe)</code>	Displays the dataframe output in the notebook.
8	(blank line)	Blank line. It separates blocks so the code is easier to read.
9	<code>plt.figure(figsize=(9, 5))</code>	Creates a new chart area with the chosen figure size.
10	<code>plt.bar(decision_counts_dataframe["india_decision_label"], decision_counts_dataframe["rows"])</code>	Draws a bar chart from the selected dataframe columns.
11	<code>plt.title("India Feasibility Decision Labels")</code>	Adds the chart title.
12	<code>plt.xlabel("Decision label")</code>	Adds the x-axis label.
13	<code>plt.ylabel("Rows")</code>	Adds the y-axis label.
14	<code>plt.xticks(rotation=35)</code>	Rotates or labels the x-axis tick text so it is readable.
15	<code>plt.grid(axis="y", alpha=0.25)</code>	Adds a light grid to make chart values easier to compare.
16	<code>plt.show()</code>	Shows the chart output in the notebook.

Cell 7: Code

Purpose: Shows the family makeup of the final shortlist.

Important output: the final shortlist still contains multiple families, so the DSS is not locked to only one chemistry.

First line: `final_shortlist_dataframe = read_project_data("final india battery shortlist.csv")`

Line	Code	Simple explanation
1	<code>final_shortlist_dataframe = read_project_data("final india battery shortlist.csv")</code>	Loads the file final india battery shortlist.csv into a dataframe.
2	(blank line)	Blank line. It separates blocks so the code is easier to read.
3	<code>final_family_count_series = final_shortlist_dataframe["battery_family"].value_counts()</code>	Counts how many rows belong to each category.
4	<code>final_family_counts_dataframe = final_family_count_series.reset_index()</code>	Converts a counted Series back into a normal dataframe.
5	<code>final_family_counts_dataframe.columns = ["battery_family", "rows"]</code>	Renames the columns so the output table is easier to understand.
6	(blank line)	Blank line. It separates blocks so the code is easier to read.
7	<code>display(final_family_counts_dataframe)</code>	Displays the dataframe output in the notebook.
8	(blank line)	Blank line. It separates blocks so the code is easier to read.
9	<code>plt.figure(figsize=(9, 5))</code>	Creates a new chart area with the chosen figure size.
10	<code>plt.bar(final_family_counts_dataframe["battery_family"]</code>	Draws a bar chart from the selected dataframe columns.

	y"], final_family_counts_dataframe["rows"])	
11	plt.title("Final Shortlist Battery Families")	Adds the chart title.
12	plt.xlabel("Battery family")	Adds the x-axis label.
13	plt.ylabel("Rows")	Adds the y-axis label.
14	plt.xticks(rotation=35)	Rotates or labels the x-axis tick text so it is readable.
15	plt.grid(axis="y", alpha=0.25)	Adds a light grid to make chart values easier to compare.
16	plt.show()	Shows the chart output in the notebook.

Cell 8: Code

Purpose: Displays the first 10 shortlisted materials with key decision columns.

Important output: this is an early look at candidate materials before the final DSS ranking.

First line: final_shortlist_dataframe = read_project_data("final india battery shortlist.csv")

Line	Code	Simple explanation
1	final_shortlist_dataframe = read_project_data("final india battery shortlist.csv")	Loads the file final india battery shortlist.csv into a dataframe.
2	(blank line)	Blank line. It separates blocks so the code is easier to read.
3	top_candidate_columns = [	Starts a Python list. The next lines are items inside that list.
4	"material_id",	A text item inside a Python list, usually a column name or feature name.
5	"formula",	A text item inside a Python list, usually a column name or feature name.
6	"battery_family",	A text item inside a Python list, usually a column name or feature name.
7	"india_decision_label",	A text item inside a Python list, usually a column name or feature name.
8	"shortlist_rule_type",	A text item inside a Python list, usually a column name or feature name.
9	"shortlist_score",	A text item inside a Python list, usually a column name or feature name.
10	"predicted_stable_probability",	A text item inside a Python list, usually a column name or feature name.
11	"predicted_energy_above_hull_clipped",	A text item inside a Python list, usually a column name or feature name.
12	"india_feasibility_score",	A text item inside a Python list, usually a column name or feature name.
13	]	Closes the current list, dictionary, or function call.
14	top_candidates_dataframe = final_shortlist_dataframe[top_candidate_columns].head(10)	Keeps only the first few rows for a short display table.
15	display(top_candidates_dataframe)	Displays the dataframe output in the notebook.

Cell 9: Markdown

Purpose: Explains that the final output is a compound ranking, not only family names.

First line: ## DSS Compound Recommendation Ranking

Line	Notebook text	Meaning
1	## DSS Compound Recommendation Ranking	Section heading. It tells the reader what the next part is about.
2	(blank line)	Blank line. It gives breathing space between text blocks.
3	The project is used as a Decision Support System here. The main output is a	Explanation text written for the presentation audience.
4	ranked list of exact lithium compound formulas. Battery family is shown only as	Explanation text written for the presentation audience.
5	supporting context.	Explanation text written for the presentation audience.
6	(blank line)	Blank line. It gives breathing space between text blocks.
7	In this flow, QML gives the first stability signal. XGBoost is used as a	Explanation text written for the presentation audience.
8	corrective backup when the QML probability is uncertain or when both models	Explanation text written for the presentation audience.
9	disagree. This keeps the DSS quantum-led without ignoring the stronger	Explanation text written for the presentation audience.
10	classical benchmark.	Explanation text written for the presentation audience.

Cell 10: Code

Purpose: Displays the compound ranking and family context ranking.

Important output: this is the project recommendation table. It ranks exact compound formulas.

First line: `dss_compound_ranking_dataframe = read_project_data("dss compound recommendation ranking.csv")`

Line	Code	Simple explanation
1	<code>dss_compound_ranking_dataframe = read_project_data("dss compound recommendation ranking.csv")</code>	Loads the file dss compound recommendation ranking.csv into a dataframe.
2	<code>dss_family_ranking_dataframe = read_project_data("dss battery family recommendation ranking.csv")</code>	Loads the file dss battery family recommendation ranking.csv into a dataframe.
3	(blank line)	Blank line. It separates blocks so the code is easier to read.
4	<code>display(dss_compound_ranking_dataframe.head(10))</code>	Displays the dataframe output in the notebook.
5	<code>display(dss_family_ranking_dataframe.head(10))</code>	Displays the dataframe output in the notebook.

Cell 11: Markdown

Purpose: Summarizes the quantum setup in presentation language.

First line: `## Quantum Setup Used`

Line	Notebook text	Meaning
------	---------------	---------

1	## Quantum Setup Used	Section heading. It tells the reader what the next part is about.
2	(blank line)	Blank line. It gives breathing space between text blocks.
3	This is explanation for the presentation, not model code.	Explanation text written for the presentation audience.
4	(blank line)	Blank line. It gives breathing space between text blocks.
5	Item Value	A row in a Markdown table.
6	--- ---	A row in a Markdown table.
7	QML method Qiskit Statevector quantum kernel classifier	A row in a Markdown table.
8	Qiskit objects `QuantumCircuit` and `Statevector.from_instruction`	A row in a Markdown table.
9	Main circuit gates `RY` rotation gates and `CP` controlled-phase gates	A row in a Markdown table.
10	Best simple QML tuning 8 qubits, `pi / 2` angle scale, SVM `C = 1.0`	A row in a Markdown table.
11	Best final QML section 4-qubit `entangled_pi` kernel with SVM `C = 5.0`	A row in a Markdown table.
12	Validation 4-fold tuning and 10 repeated train/test splits	A row in a Markdown table.
13	(blank line)	Blank line. It gives breathing space between text blocks.
14	The actual Qiskit circuit is shown in the next code cell.	Explanation text written for the presentation audience.

Cell 12: Code

Purpose: Builds and displays an actual Qiskit quantum circuit and its statevector size.

Important output: the Qiskit circuit diagram and the statevector length of 16 for 4 qubits.

First line: sample_feature_names = [

Line	Code	Simple explanation
1	sample_feature_names = [	Starts a Python list. The next lines are items inside that list.
2	"formation_energy_per_atom",	A text item inside a Python list, usually a column name or feature name.
3	"has_o",	A text item inside a Python list, usually a column name or feature name.
4	"space_group_number",	A text item inside a Python list, usually a column name or feature name.
5	"theoretical",	A text item inside a Python list, usually a column name or feature name.
6	]	Closes the current list, dictionary, or function call.

7	(blank line)	Blank line. It separates blocks so the code is easier to read.
8	sample_feature_values = [0.20, 1.00, 0.53, 1.00]	Stores four scaled example feature values that will be encoded into a 4-qubit circuit.
9	(blank line)	Blank line. It separates blocks so the code is easier to read.
10	qiskit_example_dataframe = pd.DataFrame({	Starts building a pandas dataframe so the notebook can show the values as a clean table.
11	{	Starts a dictionary. In pandas table building, a dictionary gives column names and values for one row.
12	"feature": sample_feature_names,	Sets the feature field in a dictionary row for a dataframe.
13	"scaled_value_used_in_example": sample_feature_values,	Sets the scaled_value_used_in_example field in a dictionary row for a dataframe.
14	}	Closes the current list, dictionary, or function call.
15	)	Closes the current list, dictionary, dataframe, or function call.
16	display(qiskit_example_dataframe)	Displays the dataframe output in the notebook.
17	(blank line)	Blank line. It separates blocks so the code is easier to read.
18	angle_scale_value = 3.141592653589793	Stores pi as the angle scale. Each feature value is multiplied by this number.
19	entanglement_strength = 3.141592653589793	Stores pi as the controlled-phase strength for the pairwise quantum interaction.
20	(blank line)	Blank line. It separates blocks so the code is easier to read.
21	qiskit_circuit = QuantumCircuit(4)	Creates a Qiskit circuit with 4 qubits, one qubit for each selected feature.
22	(blank line)	Blank line. It separates blocks so the code is easier to read.
23	for feature_index in range(4):	Loops through the four features to add one rotation gate to each qubit.
24	feature_value = sample_feature_values[feature_index]	Takes the current scaled feature value from the example feature list.
25	angle = angle_scale_value * feature_value	Calculates the rotation angle: angle scale multiplied by the feature value.
26	qiskit_circuit.ry(angle, feature_index)	Adds an RY rotation gate. This is how one numeric feature is encoded into one qubit.
27	(blank line)	Blank line. It separates blocks so the code is easier to read.
28	for feature_index in range(3):	Loops through neighboring qubit pairs: 0-1, 1-2, and 2-3.
29	first_value = sample_feature_values[feature_index]	Stores the first feature value in the current neighboring pair.
30	second_value = sample_feature_values[feature_index + 1]	Stores the second feature value in the current neighboring pair.
31	phase_angle = entanglement_strength * first_value * second_value	Calculates the controlled phase angle using the product of two neighboring feature values.
32	qiskit_circuit.cp(phase_angle, feature_index, feature_index + 1)	Adds a controlled-phase gate. This creates interaction between neighboring qubits.
33	(blank line)	Blank line. It separates blocks so the code is easier to read.

34	qiskit_circuit_diagram = qiskit_circuit.draw(output="mpl")	Draws the Qiskit circuit as a clean matplotlib diagram instead of terminal-style text.
35	display(qiskit_circuit_diagram)	Displays the dataframe output in the notebook.
36	(blank line)	Blank line. It separates blocks so the code is easier to read.
37	qiskit_statevector = Statevector.from_instruction(qiskit_circuit)	Simulates the quantum circuit and gets the final statevector.
38	statevector_summary_dataframe = pd.DataFrame([	Creates a small table manually from rows written in the notebook.
39	{ "statevector_length": len(qiskit_statevector.data)}	Creates one row of a manually built table using column names and values.
40	])	Closes the current list, dictionary, or function call.
41	display(statevector_summary_dataframe)	Displays the dataframe output in the notebook.

Cell 13: Markdown

Purpose: Introduces the exhaustive QML hyperparameter and feature-combination search.

First line: `## Exhaustive QML Feature-Combination Tuning`

Line	Notebook text	Meaning
1	<code>## Exhaustive QML Feature-Combination Tuning</code>	Section heading. It tells the reader what the next part is about.
2	(blank line)	Blank line. It gives breathing space between text blocks.
3	This step tests the stronger tuning approach: all selected feature-count	Explanation text written for the presentation audience.
4	combinations are compared with all angle scales and all SVM 'C' values. The	Explanation text written for the presentation audience.
5	full result table has 8,388 rows.	Explanation text written for the presentation audience.

Cell 14: Code

Purpose: Displays the best exhaustive QML tuning results.

Important output: exhaustive tuning tested 8,388 configurations and saved the best rows.

First line: `qml_exhaustive_results_dataframe = read_project_data("qml exhaustive feature combination results.csv")`

Line	Code	Simple explanation
1	<code>qml_exhaustive_results_dataframe = read_project_data("qml exhaustive feature combination results.csv")</code>	Loads the file qml exhaustive feature combination results.csv into a dataframe.
2	<code>qml_exhaustive_top_results_dataframe = read_project_data("qml exhaustive feature combination top results.csv")</code>	Loads the file qml exhaustive feature combination top results.csv into a dataframe.

3	<code>qml_exhaustive_best_result = qml_exhaustive_top_results_dataframe.iloc[0]</code>	Takes the first row after sorting. In this notebook, that row is treated as the best result.
4	<code>(blank line)</code>	Blank line. It separates blocks so the code is easier to read.
5	<code>qml_exhaustive_summary_dataframe = pd.DataFrame([</code>	Creates a small table manually from rows written in the notebook.
6	<code>{</code>	Starts a dictionary. In pandas table building, a dictionary gives column names and values for one row.
7	<code>"total_configurations": len(qml_exhaustive_results_dataframe),</code>	Sets the total_configurations field in a dictionary row for a dataframe.
8	<code>"feature_combinations": 466,</code>	Sets the feature_combinations field in a dictionary row for a dataframe.
9	<code>"feature_count": int(qml_exhaustive_best_result["feature_count"]),</code>	Sets the feature_count field in a dictionary row for a dataframe.
10	<code>"angle_scale": qml_exhaustive_best_result["angle_scale"],</code>	Sets the angle_scale field in a dictionary row for a dataframe.
11	<code>"svm_c": qml_exhaustive_best_result["c_value"],</code>	Sets the svm_c field in a dictionary row for a dataframe.
12	<code>"cv_accuracy": qml_exhaustive_best_result["cv_accuracy"],</code>	Sets the cv_accuracy field in a dictionary row for a dataframe.
13	<code>"cv_stable_f1": qml_exhaustive_best_result["cv_stable_f1"],</code>	Sets the cv_stable_f1 field in a dictionary row for a dataframe.
14	<code>}</code>	Closes the current list, dictionary, or function call.
15	<code>])</code>	Closes the current list, dictionary, or function call.
16	<code>display(qml_exhaustive_summary_dataframe)</code>	Displays the dataframe output in the notebook.
17	<code>(blank line)</code>	Blank line. It separates blocks so the code is easier to read.
18	<code>qml_exhaustive_top_display_dataframe = qml_exhaustive_top_results_dataframe[</code>	Starts selecting only the useful columns from the top QML tuning results.
19	<code>[</code>	Starts a list. In this notebook, the list is usually a group of columns, metrics, or model rows.
20	<code>"feature_count",</code>	A text item inside a Python list, usually a column name or feature name.
21	<code>"feature_names",</code>	A text item inside a Python list, usually a column name or feature name.
22	<code>"angle_scale",</code>	A text item inside a Python list, usually a column name or feature name.
23	<code>"c_value",</code>	A text item inside a Python list, usually a column name or feature name.
24	<code>"cv_accuracy",</code>	A text item inside a Python list, usually a column name or feature name.
25	<code>"cv_stable_recall",</code>	A text item inside a Python list, usually a column name or feature name.
26	<code>"cv_stable_f1",</code>	A text item inside a Python list, usually a column name or feature name.
27	<code>]</code>	Closes the current list, dictionary, or function call.
28	<code>].head(10)</code>	Finishes the column selection and keeps only the first few rows for display.
29	<code>display(qml_exhaustive_top_display_dataframe)</code>	Displays the dataframe output in the notebook.

Cell 15: Code

Purpose: Compares model metrics across QML variants and XGBoost.

Important output: the table compares accuracy, precision, recall, and F1 for all main models.

First line: `qml_predictions_dataframe = read_project_data("qml baseline predictions.csv")`

Line	Code	Simple explanation
1	<code>qml_predictions_dataframe = read_project_data("qml baseline predictions.csv")</code>	Loads the file qml baseline predictions.csv into a dataframe.
2	<code>tuned_qml_predictions_dataframe = read_project_data("qml tuned best predictions.csv")</code>	Loads the file qml tuned best predictions.csv into a dataframe.
3	<code>improved_qml_predictions_dataframe = read_project_data("improved qml best predictions.csv")</code>	Loads the file improved qml best predictions.csv into a dataframe.
4	<code>improved_qml_threshold_predictions_dataframe = read_project_data("improved qml threshold predictions.csv")</code>	Loads the file improved qml threshold predictions.csv into a dataframe.
5	<code>improved_qml_alignment_predictions_dataframe = read_project_data("improved qml alignment predictions.csv")</code>	Loads the file improved qml alignment predictions.csv into a dataframe.
6	(blank line)	Blank line. It separates blocks so the code is easier to read.
7	<code>tuned_true_labels = tuned_qml_predictions_dataframe["target_is_stable"]</code>	Selects the true stability labels. These are the answers the model should predict.
8	<code>tuned_qml_predicted_labels = tuned_qml_predictions_dataframe["tuned_qml_predicted_label"]</code>	Selects the predicted labels made by the model.
9	<code>improved_true_labels = improved_qml_predictions_dataframe["target_is_stable"]</code>	Selects the true stability labels. These are the answers the model should predict.
10	<code>improved_qml_predicted_labels = improved_qml_predictions_dataframe["improved_qml_predicted_label"]</code>	Selects the predicted labels made by the model.
11	<code>threshold_true_labels = improved_qml_threshold_predictions_dataframe["target_is_stable"]</code>	Selects the true stability labels. These are the answers the model should predict.
12	<code>threshold_qml_predicted_labels = improved_qml_threshold_predictions_dataframe["threshold_qml_predicted_label"]</code>	Selects the predicted labels made by the model.
13	<code>alignment_true_labels = improved_qml_alignment_predictions_dataframe["target_is_stable"]</code>	Selects the true stability labels. These are the answers the model should predict.
14	<code>alignment_qml_predicted_labels = improved_qml_alignment_predictions_dataframe["alignment_qml_predicted_label"]</code>	Selects the predicted labels made by the model.
15	(blank line)	Blank line. It separates blocks so the code is easier to read.
16	<code>def calculate_model_scores(model_name, true_values, predicted_values):</code>	Defines a helper function that calculates all important classification scores for one model.
17	<code>return {</code>	Returns a dictionary containing the model name and its scores.

18	"model": model_name,	Sets the model field in a dictionary row for a dataframe.
19	"accuracy": accuracy_score(true_values, predicted_values),	Calculates accuracy, meaning the fraction of predictions that were correct.
20	"stable_precision": precision_score(true_values, predicted_values, zero_division=0),	Calculates precision for stable materials: when the model says stable, how often it is right.
21	"stable_recall": recall_score(true_values, predicted_values, zero_division=0),	Calculates recall for stable materials: out of all truly stable materials, how many were found.
22	"stable_f1": f1_score(true_values, predicted_values, zero_division=0),	Calculates F1 score, a balance between precision and recall.
23	}	Closes the current list, dictionary, or function call.
24	(blank line)	Blank line. It separates blocks so the code is easier to read.
25	model_score_inputs = [	Starts a Python list. The next lines are items inside that list.
26	["QML quantum kernel", qml_predictions_dataframe["target_is_stable"], qml_predictions_dataframe["qml_predicted_label"]],	Adds one QML model row to the list of models that will be scored.
27	["Tuned QML quantum kernel", tuned_qml_predictions_dataframe["target_is_stable"], tuned_qml_predictions_dataframe["tuned_qml_predicted_label"]],	Adds one QML model row to the list of models that will be scored.
28	["Improved QML separate section", improved_qml_predictions_dataframe["target_is_stable"], improved_qml_predictions_dataframe["improved_qml_predicted_label"]],	Adds one QML model row to the list of models that will be scored.
29	["Improved QML threshold tuning", improved_qml_threshold_predictions_dataframe["target_is_stable"], improved_qml_threshold_predictions_dataframe["threshold_qml_predicted_label"]],	Adds one QML model row to the list of models that will be scored.
30	["Improved QML kernel alignment", improved_qml_alignment_predictions_dataframe["target_is_stable"], improved_qml_alignment_predictions_dataframe["alignment_qml_predicted_label"]],	Adds one QML model row to the list of models that will be scored.
31	["XGBoost same QML data", qml_predictions_dataframe["target_is_stable"], qml_predictions_dataframe["xgboost_same_data_predicted_label"]],	Adds one QML model row to the list of models that will be scored.
32	]	Closes the current list, dictionary, or function call.
33	(blank line)	Blank line. It separates blocks so the code is easier to read.
34	metric_rows = []	Creates an empty list that will collect one score row per model.
35	for model_name, true_values, predicted_values in model_score_inputs:	Loops through each model and its true/predicted labels.
36	metric_rows.append(calculate_model_scores(model_name, true_values, predicted_values))	Adds one model score row to the metrics list.
37	(blank line)	Blank line. It separates blocks so the code is easier to read.

38	<code>metric_rows.append({</code>	Adds one model score row to the metrics list.
39	<code>"model": "XGBoost full project",</code>	A text item inside a Python list, usually a column name or feature name.
40	<code>"accuracy": 0.9091,</code>	Sets the accuracy field in a dictionary row for a dataframe.
41	<code>"stable_precision": 0.7300,</code>	Sets the stable_precision field in a dictionary row for a dataframe.
42	<code>"stable_recall": 0.7000,</code>	Sets the stable_recall field in a dictionary row for a dataframe.
43	<code>"stable_f1": 0.7100,</code>	Sets the stable_f1 field in a dictionary row for a dataframe.
44	<code>})</code>	Completes the current dataframe/list/scoring step in this cell.
45	(blank line)	Blank line. It separates blocks so the code is easier to read.
46	<code>metric_dataframe = pd.DataFrame(metric_rows).round(4)</code>	Converts the collected metric rows into a dataframe and rounds the numbers.
47	<code>display(metric_dataframe)</code>	Displays the dataframe output in the notebook.
48	(blank line)	Blank line. It separates blocks so the code is easier to read.
49	<code>metric_plot_dataframe = metric_dataframe.set_index("model")</code>	Uses the model name as the chart index, so each model becomes one bar group.
50	<code>metric_plot_dataframe = metric_plot_dataframe[["accuracy", "stable_precision", "stable_recall", "stable_f1"]]</code>	Completes the current dataframe/list/scoring step in this cell.
51	<code>metric_plot_dataframe.plot(kind="bar", figsize=(9, 5))</code>	Draws the model scores as a bar chart.
52	<code>plt.title("Model Performance Comparison")</code>	Adds the chart title.
53	<code>plt.ylabel("Score")</code>	Adds the y-axis label.
54	<code>plt.ylim(0, 1.05)</code>	Completes the current dataframe/list/scoring step in this cell.
55	<code>plt.xticks(rotation=15)</code>	Rotates or labels the x-axis tick text so it is readable.
56	<code>plt.grid(axis="y", alpha=0.25)</code>	Adds a light grid to make chart values easier to compare.
57	<code>plt.show()</code>	Shows the chart output in the notebook.
58	(blank line)	Blank line. It separates blocks so the code is easier to read.

Cell 16: Code

Purpose: Builds a confusion matrix for the tuned QML model.

Important output: the confusion matrix shows where tuned QML was right or wrong.

First line: `qml_confusion_matrix = confusion_matrix(tuned_true_labels, tuned_qml_predicted_labels, labels=[0, 1])`

Line	Code	Simple explanation
1	<code>qml_confusion_matrix = confusion_matrix(tuned_true_labels, tuned_qml_predicted_labels, labels=[0, 1])</code>	Builds a confusion matrix, showing right and wrong predictions by class.

2	<code>confusion_dataframe = pd.DataFrame(</code>	Starts building a pandas dataframe so the notebook can show the values as a clean table.
3	<code>qml_confusion_matrix,</code>	Completes the current dataframe/list/scoring step in this cell.
4	<code>index=["actual_unstable", "actual_stable"],</code>	Names the rows of the displayed table.
5	<code>columns=["predicted_unstable", "predicted_stable"],</code>	Names the columns of the displayed table.
6	<code>)</code>	Closes the current list, dictionary, dataframe, or function call.
7	<code>display(confusion_dataframe)</code>	Displays the dataframe output in the notebook.
8	(blank line)	Blank line. It separates blocks so the code is easier to read.
9	<code>plt.figure(figsize=(5.8, 4.8))</code>	Creates a new chart area with the chosen figure size.
10	<code>plt.imshow(qml_confusion_matrix, cmap="Blues")</code>	Shows the confusion matrix as a colored heatmap.
11	<code>plt.title("Tuned QML Confusion Matrix")</code>	Adds the chart title.
12	<code>plt.xticks([0, 1], ["Predicted unstable", "Predicted stable"])</code>	Rotates or labels the x-axis tick text so it is readable.
13	<code>plt.yticks([0, 1], ["Actual unstable", "Actual stable"])</code>	Completes the current dataframe/list/scoring step in this cell.
14	<code>for row_index in range(2):</code>	Loops through the two rows of the confusion matrix.
15	<code>for column_index in range(2):</code>	Loops through the two columns of the confusion matrix.
16	<code>cell_value = qml_confusion_matrix[row_index, column_index]</code>	Reads the number inside the current confusion-matrix cell.
17	<code>plt.text(column_index, row_index, cell_value, ha="center", va="center")</code>	Writes the number directly inside the heatmap cell.
18	<code>plt.colorbar()</code>	Adds the color scale beside the heatmap.
19	<code>plt.show()</code>	Shows the chart output in the notebook.

Cell 17: Code

Purpose: Shows sample predictions from the tuned QML model.

First line: `tuned_qml_predictions_dataframe = read_project_data("qml tuned best predictions.csv")`

Line	Code	Simple explanation
1	<code>tuned_qml_predictions_dataframe = read_project_data("qml tuned best predictions.csv")</code>	Loads the file qml tuned best predictions.csv into a dataframe.
2	(blank line)	Blank line. It separates blocks so the code is easier to read.
3	<code>sample_predictions_dataframe = tuned_qml_predictions_dataframe[</code>	Starts selecting the useful sample-prediction columns.
4	<code>[</code>	Starts a list. In this notebook, the list is usually a group of columns, metrics, or model rows.
5	<code>"material_id",</code>	A text item inside a Python list, usually a column name or feature name.

6	"formula",	A text item inside a Python list, usually a column name or feature name.
7	"target_is_stable",	A text item inside a Python list, usually a column name or feature name.
8	"tuned_qml_predicted_label",	A text item inside a Python list, usually a column name or feature name.
9	"tuned_qml_stable_probability",	A text item inside a Python list, usually a column name or feature name.
10	]	Closes the current list, dictionary, or function call.
11	].head(10)	Finishes the column selection and keeps only the first few rows for display.
12	display(sample_predictions_dataframe)	Displays the dataframe output in the notebook.

Cell 18: Code

Purpose: Summarizes the improved QML section using feature importance and PCA.

Important output: the improved QML section gives a separate QML experiment, not a replacement for the earlier one.

First line: `improved_qml_tuning_results_dataframe = read_project_data("improved qml tuning results.csv")`

Line	Code	Simple explanation
1	<code>improved_qml_tuning_results_dataframe = read_project_data("improved qml tuning results.csv")</code>	Loads the file improved qml tuning results.csv into a dataframe.
2	<code>improved_qml_predictions_dataframe = read_project_data("improved qml best predictions.csv")</code>	Loads the file improved qml best predictions.csv into a dataframe.
3	(blank line)	Blank line. It separates blocks so the code is easier to read.
4	<code>sorted_improved_results_dataframe = improved_qml_tuning_results_dataframe.sort_values(</code>	Sorts the result table so the best model setting comes first.
5	<code>by=["cv_stable_f1", "cv_accuracy", "cv_stable_recall"],</code>	Chooses which score columns decide the sorting order.
6	<code>ascending=[False, False, False],</code>	Says whether each sorting column should go from low to high or high to low.
7	<code>)</code>	Closes the current list, dictionary, dataframe, or function call.
8	<code>improved_best_result = sorted_improved_results_dataframe.iloc[0]</code>	Takes the first row after sorting. In this notebook, that row is treated as the best result.
9	(blank line)	Blank line. It separates blocks so the code is easier to read.
10	<code>improved_true_labels = improved_qml_predictions_dataframe["target_is_stable"]</code>	Selects the true stability labels. These are the answers the model should predict.
11	<code>improved_qml_predicted_labels = improved_qml_predictions_dataframe["improved_qml_predicted_label"]</code>	Selects the predicted labels made by the model.
12	(blank line)	Blank line. It separates blocks so the code is easier to read.
13	<code>improved_test_accuracy = accuracy_score(improved_true_labels, improved_qml_predicted_labels)</code>	Calculates accuracy, meaning the fraction of predictions that were correct.

14	<code>improved_test_f1 = f1_score(improved_true_labels, improved_qml_predicted_labels, zero_division=0)</code>	Calculates F1 score, a balance between precision and recall.
15	(blank line)	Blank line. It separates blocks so the code is easier to read.
16	<code>improved_summary_row = {</code>	Starts a dictionary that will become one summary row in the output table.
17	<code>"section": "Improved QML separate section",</code>	A text item inside a Python list, usually a column name or feature name.
18	<code>"feature_preparation": "feature importance + PCA",</code>	A text item inside a Python list, usually a column name or feature name.
19	<code>"best_kernel": improved_best_result["kernel_name"],</code>	Sets the best_kernel field in a dictionary row for a dataframe.
20	<code>"best_qubits": int(improved_best_result["pca_component_count"]),</code>	Sets the best_qubits field in a dictionary row for a dataframe.
21	<code>"angle_scale": improved_best_result["angle_scale"],</code>	Sets the angle_scale field in a dictionary row for a dataframe.
22	<code>"svm_c": improved_best_result["c_value"],</code>	Sets the svm_c field in a dictionary row for a dataframe.
23	<code>"cv_stable_f1": improved_best_result["cv_stable_f1"],</code>	Sets the cv_stable_f1 field in a dictionary row for a dataframe.
24	<code>"test_accuracy": improved_test_accuracy,</code>	Sets the test_accuracy field in a dictionary row for a dataframe.
25	<code>"test_stable_f1": improved_test_f1,</code>	Sets the test_stable_f1 field in a dictionary row for a dataframe.
26	<code>}</code>	Closes the current list, dictionary, or function call.
27	(blank line)	Blank line. It separates blocks so the code is easier to read.
28	<code>improved_summary_dataframe = pd.DataFrame([improved_summary_row])</code>	Starts building a pandas dataframe so the notebook can show the values as a clean table.
29	<code>improved_summary_dataframe = improved_summary_dataframe.round(4)</code>	Rounds the number so the table is easier to read.
30	(blank line)	Blank line. It separates blocks so the code is easier to read.
31	<code>display(improved_summary_dataframe)</code>	Displays the dataframe output in the notebook.

Cell 19: Code

Purpose: Builds a confusion matrix for the improved QML section.

First line: `improved_qml_confusion_matrix = confusion_matrix(`

Line	Code	Simple explanation
1	<code>improved_qml_confusion_matrix = confusion_matrix(</code>	Builds a confusion matrix, showing right and wrong predictions by class.
2	<code>improved_true_labels,</code>	Stores the actual labels for the improved QML test rows.
3	<code>improved_qml_predicted_labels,</code>	Stores the improved QML predicted labels.
4	<code>labels=[0, 1],</code>	Completes the current dataframe/list/scoring step in this cell.
5	<code>)</code>	Closes the current list, dictionary, dataframe, or function call.
6	<code>improved_confusion_dataframe = pd.DataFrame(</code>	Starts building a pandas dataframe so the notebook can show the values as

		a clean table.
7	<code>improved_qml_confusion_matrix,</code>	Completes the current dataframe/list/scoring step in this cell.
8	<code>index=["actual_unstable", "actual_stable"],</code>	Names the rows of the displayed table.
9	<code>columns=["predicted_unstable", "predicted_stable"],</code>	Names the columns of the displayed table.
10	<code>)</code>	Closes the current list, dictionary, dataframe, or function call.
11	<code>display(improved_confusion_dataframe)</code>	Displays the dataframe output in the notebook.
12	(blank line)	Blank line. It separates blocks so the code is easier to read.
13	<code>plt.figure(figsize=(5.8, 4.8))</code>	Creates a new chart area with the chosen figure size.
14	<code>plt.imshow(improved_qml_confusion_matrix, cmap="Blues")</code>	Shows the confusion matrix as a colored heatmap.
15	<code>plt.title("Improved QML Confusion Matrix")</code>	Adds the chart title.
16	<code>plt.xticks([0, 1], ["Predicted unstable", "Predicted stable"])</code>	Rotates or labels the x-axis tick text so it is readable.
17	<code>plt.yticks([0, 1], ["Actual unstable", "Actual stable"])</code>	Completes the current dataframe/list/scoring step in this cell.
18	<code>for row_index in range(2):</code>	Loops through the two rows of the confusion matrix.
19	<code>for column_index in range(2):</code>	Loops through the two columns of the confusion matrix.
20	<code>cell_value = improved_qml_confusion_matrix[row_index, column_index]</code>	Reads the number inside the current confusion-matrix cell.
21	<code>plt.text(column_index, row_index, cell_value, ha="center", va="center")</code>	Writes the number directly inside the heatmap cell.
22	<code>plt.colorbar()</code>	Adds the color scale beside the heatmap.
23	<code>plt.show()</code>	Shows the chart output in the notebook.

Cell 20: Code

Purpose: Shows sample predictions from the improved QML section.

First line: `improved_qml_predictions_dataframe = read_project_data("improved qml best predictions.csv")`

Line	Code	Simple explanation
1	<code>improved_qml_predictions_dataframe = read_project_data("improved qml best predictions.csv")</code>	Loads the file improved qml best predictions.csv into a dataframe.
2	(blank line)	Blank line. It separates blocks so the code is easier to read.
3	<code>improved_sample_predictions_dataframe = improved_qml_predictions_dataframe[</code>	Starts selecting the useful sample-prediction columns.
4	<code>[</code>	Starts a list. In this notebook, the list is usually a group of columns, metrics, or model rows.

5	"material_id",	A text item inside a Python list, usually a column name or feature name.
6	"formula",	A text item inside a Python list, usually a column name or feature name.
7	"target_is_stable",	A text item inside a Python list, usually a column name or feature name.
8	"improved_qml_predicted_label",	A text item inside a Python list, usually a column name or feature name.
9	"improved_qml_stable_probability",	A text item inside a Python list, usually a column name or feature name.
10	]	Closes the current list, dictionary, or function call.
11	].head(10)	Finishes the column selection and keeps only the first few rows for display.
12	display(improved_sample_predictions_dataframe)	Displays the dataframe output in the notebook.

Cell 21: Code

Purpose: Summarizes the threshold-tuned QML experiment.

Important output: threshold tuning changes the stable probability cutoff to improve stable-material detection.

First line: improved_qml_threshold_results_dataframe = read_project_data("improved qml threshold results.csv")

Line	Code	Simple explanation
1	improved_qml_threshold_results_dataframe = read_project_data("improved qml threshold results.csv")	Loads the file improved qml threshold results.csv into a dataframe.
2	improved_qml_threshold_predictions_dataframe = read_project_data("improved qml threshold predictions.csv")	Loads the file improved qml threshold predictions.csv into a dataframe.
3	(blank line)	Blank line. It separates blocks so the code is easier to read.
4	sorted_threshold_results_dataframe = improved_qml_th reshold_results_dataframe.sort_values(	Sorts the result table so the best model setting comes first.
5	by=["cv_stable_f1", "cv_accuracy", "cv_stable_recall"],	Chooses which score columns decide the sorting order.
6	ascending=[False, False, False],	Says whether each sorting column should go from low to high or high to low.
7	)	Closes the current list, dictionary, dataframe, or function call.
8	improved_best_threshold_result = sorted_threshold_results_dataframe.iloc[0]	Takes the first row after sorting. In this notebook, that row is treated as the best result.
9	(blank line)	Blank line. It separates blocks so the code is easier to read.
10	threshold_true_labels = improved_qml_threshold_predi ctions_dataframe["target_is_stable"]	Selects the true stability labels. These are the answers the model should predict.
11	threshold_qml_predicted_labels = improved_qml_thresh old_predictions_dataframe["threshold_qml_predicted_l abel"]	Selects the predicted labels made by the model.
12	(blank line)	Blank line. It separates blocks so the code is easier to read.

13	<code>threshold_test_accuracy = accuracy_score(threshold_true_labels, threshold_qml_predicted_labels)</code>	Calculates accuracy, meaning the fraction of predictions that were correct.
14	<code>threshold_test_precision = precision_score(threshold_true_labels, threshold_qml_predicted_labels, zero_division=0)</code>	Calculates precision for stable materials: when the model says stable, how often it is right.
15	<code>threshold_test_recall = recall_score(threshold_true_labels, threshold_qml_predicted_labels, zero_division=0)</code>	Calculates recall for stable materials: out of all truly stable materials, how many were found.
16	<code>threshold_test_f1 = f1_score(threshold_true_labels, threshold_qml_predicted_labels, zero_division=0)</code>	Calculates F1 score, a balance between precision and recall.
17	<code>(blank line)</code>	Blank line. It separates blocks so the code is easier to read.
18	<code>threshold_summary_row = {</code>	Starts a dictionary that will become one summary row in the output table.
19	<code>"section": "Threshold experiment",</code>	A text item inside a Python list, usually a column name or feature name.
20	<code>"selected_threshold": improved_best_threshold_result["stable_threshold"],</code>	Sets the <code>selected_threshold</code> field in a dictionary row for a dataframe.
21	<code>"cv_stable_f1": improved_best_threshold_result["cv_stable_f1"],</code>	Sets the <code>cv_stable_f1</code> field in a dictionary row for a dataframe.
22	<code>"test_accuracy": threshold_test_accuracy,</code>	Sets the <code>test_accuracy</code> field in a dictionary row for a dataframe.
23	<code>"test_stable_precision": threshold_test_precision,</code>	Sets the <code>test_stable_precision</code> field in a dictionary row for a dataframe.
24	<code>"test_stable_recall": threshold_test_recall,</code>	Sets the <code>test_stable_recall</code> field in a dictionary row for a dataframe.
25	<code>"test_stable_f1": threshold_test_f1,</code>	Sets the <code>test_stable_f1</code> field in a dictionary row for a dataframe.
26	<code>}</code>	Closes the current list, dictionary, or function call.
27	<code>(blank line)</code>	Blank line. It separates blocks so the code is easier to read.
28	<code>threshold_summary_dataframe = pd.DataFrame([threshold_summary_row])</code>	Starts building a pandas dataframe so the notebook can show the values as a clean table.
29	<code>threshold_summary_dataframe = threshold_summary_dataframe.round(4)</code>	Rounds the number so the table is easier to read.
30	<code>(blank line)</code>	Blank line. It separates blocks so the code is easier to read.
31	<code>display(threshold_summary_dataframe)</code>	Displays the dataframe output in the notebook.

Cell 22: Code

Purpose: Builds a confusion matrix for the threshold-tuned QML experiment.

First line: `threshold_qml_confusion_matrix = confusion_matrix(`

Line	Code	Simple explanation
1	<code>threshold_qml_confusion_matrix = confusion_matrix(</code>	Builds a confusion matrix, showing right and wrong predictions by class.
2	<code>threshold_true_labels,</code>	Stores the actual labels for the threshold-tuned QML test rows.

3	<code>threshold_qml_predicted_labels,</code>	Stores the threshold-tuned QML predicted labels.
4	<code>labels=[0, 1],</code>	Completes the current dataframe/list/scoring step in this cell.
5	<code>)</code>	Closes the current list, dictionary, dataframe, or function call.
6	<code>threshold_confusion_dataframe = pd.DataFrame(</code>	Starts building a pandas dataframe so the notebook can show the values as a clean table.
7	<code>threshold_qml_confusion_matrix,</code>	Completes the current dataframe/list/scoring step in this cell.
8	<code>index=["actual_unstable", "actual_stable"],</code>	Names the rows of the displayed table.
9	<code>columns=["predicted_unstable", "predicted_stable"],</code>	Names the columns of the displayed table.
10	<code>)</code>	Closes the current list, dictionary, dataframe, or function call.
11	<code>display(threshold_confusion_dataframe)</code>	Displays the dataframe output in the notebook.
12	<code>(blank line)</code>	Blank line. It separates blocks so the code is easier to read.
13	<code>plt.figure(figsize=(5.8, 4.8))</code>	Creates a new chart area with the chosen figure size.
14	<code>plt.imshow(threshold_qml_confusion_matrix, cmap="Blues")</code>	Shows the confusion matrix as a colored heatmap.
15	<code>plt.title("Threshold-Tuned QML Confusion Matrix")</code>	Adds the chart title.
16	<code>plt.xticks([0, 1], ["Predicted unstable", "Predicted stable"])</code>	Rotates or labels the x-axis tick text so it is readable.
17	<code>plt.yticks([0, 1], ["Actual unstable", "Actual stable"])</code>	Completes the current dataframe/list/scoring step in this cell.
18	<code>for row_index in range(2):</code>	Loops through the two rows of the confusion matrix.
19	<code>for column_index in range(2):</code>	Loops through the two columns of the confusion matrix.
20	<code>cell_value = threshold_qml_confusion_matrix[row_index, column_index]</code>	Reads the number inside the current confusion-matrix cell.
21	<code>plt.text(column_index, row_index, cell_value, ha="center", va="center")</code>	Writes the number directly inside the heatmap cell.
22	<code>plt.colorbar()</code>	Adds the color scale beside the heatmap.
23	<code>plt.show()</code>	Shows the chart output in the notebook.

Cell 23: Code

Purpose: Summarizes the kernel-alignment QML experiment.

Important output: kernel alignment selects features based on how well the quantum kernel matches the target labels.

First line: `improved_qml_alignment_results_dataframe = read_project_data("improved qml alignment results.csv")`

Line	Code	Simple explanation
------	------	--------------------

1	<code>improved_qml_alignment_results_dataframe = read_project_data("improved qml alignment results.csv")</code>	Loads the file improved qml alignment results.csv into a dataframe.
2	<code>improved_qml_alignment_predictions_dataframe = read_project_data("improved qml alignment predictions.csv")</code>	Loads the file improved qml alignment predictions.csv into a dataframe.
3	<code>(blank line)</code>	Blank line. It separates blocks so the code is easier to read.
4	<code>sorted_alignment_results_dataframe = improved_qml_al ignment_results_dataframe.sort_values(</code>	Sorts the result table so the best model setting comes first.
5	<code>by=["cv_stable_f1", "cv_accuracy", "kernel_target_alignment"],</code>	Chooses which score columns decide the sorting order.
6	<code>ascending=[False, False, False],</code>	Says whether each sorting column should go from low to high or high to low.
7	<code>)</code>	Closes the current list, dictionary, dataframe, or function call.
8	<code>improved_best_alignment_result = sorted_alignment_results_dataframe.iloc[0]</code>	Takes the first row after sorting. In this notebook, that row is treated as the best result.
9	<code>(blank line)</code>	Blank line. It separates blocks so the code is easier to read.
10	<code>alignment_true_labels = improved_qml_alignment_predi ctions_dataframe["target_is_stable"]</code>	Selects the true stability labels. These are the answers the model should predict.
11	<code>alignment_qml_predicted_labels = improved_qml_alignm ent_predictions_dataframe["alignment_qml_predicted_l abel"]</code>	Selects the predicted labels made by the model.
12	<code>(blank line)</code>	Blank line. It separates blocks so the code is easier to read.
13	<code>alignment_test_accuracy = accuracy_score(alignment_true_labels, alignment_qml_predicted_labels)</code>	Calculates accuracy, meaning the fraction of predictions that were correct.
14	<code>alignment_test_f1 = f1_score(alignment_true_labels, alignment_qml_predicted_labels, zero_division=0)</code>	Calculates F1 score, a balance between precision and recall.
15	<code>(blank line)</code>	Blank line. It separates blocks so the code is easier to read.
16	<code>alignment_summary_row = {</code>	Starts a dictionary that will become one summary row in the output table.
17	<code>"section": "Kernel alignment experiment",</code>	A text item inside a Python list, usually a column name or feature name.
18	<code>"feature_set": improved_best_alignment_result["feature_set_name"],</code>	Sets the feature_set field in a dictionary row for a dataframe.
19	<code>"feature_count": int(improved_best_alignment_res ult["feature_count"]),</code>	Sets the feature_count field in a dictionary row for a dataframe.
20	<code>"kernel": improved_best_alignment_result["kernel_name"],</code>	Sets the kernel field in a dictionary row for a dataframe.
21	<code>"angle_scale": improved_best_alignment_result["angle_scale"],</code>	Sets the angle_scale field in a dictionary row for a dataframe.
22	<code>"svm_c": improved_best_alignment_result["c_value"],</code>	Sets the svm_c field in a dictionary row for a dataframe.
23	<code>"kernel_target_alignment": improved_best_alignme</code>	Sets the kernel_target_alignment field in a dictionary row for a dataframe.

	nt_result["kernel_target_alignment"],	
24	"cv_stable_f1": improved_best_alignment_result["cv_stable_f1"],	Sets the cv_stable_f1 field in a dictionary row for a dataframe.
25	"test_accuracy": alignment_test_accuracy,	Sets the test_accuracy field in a dictionary row for a dataframe.
26	"test_stable_f1": alignment_test_f1,	Sets the test_stable_f1 field in a dictionary row for a dataframe.
27	}	Closes the current list, dictionary, or function call.
28	(blank line)	Blank line. It separates blocks so the code is easier to read.
29	alignment_summary_dataframe = pd.DataFrame([alignment_summary_row])	Starts building a pandas dataframe so the notebook can show the values as a clean table.
30	alignment_summary_dataframe = alignment_summary_dataframe.round(4)	Rounds the number so the table is easier to read.
31	(blank line)	Blank line. It separates blocks so the code is easier to read.
32	display(alignment_summary_dataframe)	Displays the dataframe output in the notebook.

Cell 24: Code

Purpose: Builds a confusion matrix for the kernel-alignment QML experiment.

First line: alignment_qml_confusion_matrix = confusion_matrix(

Line	Code	Simple explanation
1	alignment_qml_confusion_matrix = confusion_matrix(	Builds a confusion matrix, showing right and wrong predictions by class.
2	alignment_true_labels,	Stores the actual labels for the kernel-alignment QML test rows.
3	alignment_qml_predicted_labels,	Stores the kernel-alignment QML predicted labels.
4	labels=[0, 1],	Completes the current dataframe/list/scoring step in this cell.
5	)	Closes the current list, dictionary, dataframe, or function call.
6	alignment_confusion_dataframe = pd.DataFrame(	Starts building a pandas dataframe so the notebook can show the values as a clean table.
7	alignment_qml_confusion_matrix,	Completes the current dataframe/list/scoring step in this cell.
8	index=["actual_unstable", "actual_stable"],	Names the rows of the displayed table.
9	columns=["predicted_unstable", "predicted_stable"],	Names the columns of the displayed table.
10	)	Closes the current list, dictionary, dataframe, or function call.
11	display(alignment_confusion_dataframe)	Displays the dataframe output in the notebook.
12	(blank line)	Blank line. It separates blocks so the code is easier to read.
13	plt.figure(figsize=(5.8, 4.8))	Creates a new chart area with the chosen figure size.
14	plt.imshow(alignment_qml_confusion_matrix, cmap="Blues")	Shows the confusion matrix as a colored heatmap.
15	plt.title("Kernel-Alignment QML Confusion Matrix")	Adds the chart title.

16	<code>plt.xticks([0, 1], ["Predicted unstable", "Predicted stable"])</code>	Rotates or labels the x-axis tick text so it is readable.
17	<code>plt.yticks([0, 1], ["Actual unstable", "Actual stable"])</code>	Completes the current dataframe/list/scoring step in this cell.
18	<code>for row_index in range(2):</code>	Loops through the two rows of the confusion matrix.
19	<code>for column_index in range(2):</code>	Loops through the two columns of the confusion matrix.
20	<code>cell_value = alignment_qml_confusion_matrix[row_index, column_index]</code>	Reads the number inside the current confusion-matrix cell.
21	<code>plt.text(column_index, row_index, cell_value, ha="center", va="center")</code>	Writes the number directly inside the heatmap cell.
22	<code>plt.colorbar()</code>	Adds the color scale beside the heatmap.
23	<code>plt.show()</code>	Shows the chart output in the notebook.

Cell 25: Code

Purpose: Shows sample predictions from the kernel-alignment QML experiment.

First line: `improved_qml_alignment_predictions_dataframe = read_project_data("improved qml alignment predictions.csv")`

Line	Code	Simple explanation
1	<code>improved_qml_alignment_predictions_dataframe = read_project_data("improved qml alignment predictions.csv")</code>	Loads the file improved qml alignment predictions.csv into a dataframe.
2	(blank line)	Blank line. It separates blocks so the code is easier to read.
3	<code>alignment_sample_predictions_dataframe = improved_qml_alignment_predictions_dataframe[</code>	Starts selecting the useful sample-prediction columns.
4	<code>[</code>	Starts a list. In this notebook, the list is usually a group of columns, metrics, or model rows.
5	<code>"material_id",</code>	A text item inside a Python list, usually a column name or feature name.
6	<code>"formula",</code>	A text item inside a Python list, usually a column name or feature name.
7	<code>"target_is_stable",</code>	A text item inside a Python list, usually a column name or feature name.
8	<code>"alignment_qml_predicted_label",</code>	A text item inside a Python list, usually a column name or feature name.
9	<code>"alignment_qml_stable_probability",</code>	A text item inside a Python list, usually a column name or feature name.
10	<code>]</code>	Closes the current list, dictionary, or function call.
11	<code>].head(10)</code>	Finishes the column selection and keeps only the first few rows for display.
12	<code>display(alignment_sample_predictions_dataframe)</code>	Displays the dataframe output in the notebook.

Cell 26: Code

Purpose: Summarizes the best QML setup across 10 random train/test splits.

Important output: repeated splits test whether QML result is stable across different random train/test choices.

First line: `best_qml_repeated_split_results_dataframe = read_project_data("best qml repeated split results.csv")`

Line	Code	Simple explanation
1	<code>best_qml_repeated_split_results_dataframe = read_project_data("best qml repeated split results.csv")</code>	Loads the file best qml repeated split results.csv into a dataframe.
2	(blank line)	Blank line. It separates blocks so the code is easier to read.
3	<code>repeated_split_summary_rows = []</code>	Starts an empty list for summary statistics across repeated train/test splits.
4	<code>repeated_split_metric_columns = [</code>	Starts a Python list. The next lines are items inside that list.
5	<code>"accuracy",</code>	A text item inside a Python list, usually a column name or feature name.
6	<code>"stable_precision",</code>	A text item inside a Python list, usually a column name or feature name.
7	<code>"stable_recall",</code>	A text item inside a Python list, usually a column name or feature name.
8	<code>"stable_f1",</code>	A text item inside a Python list, usually a column name or feature name.
9	<code>]</code>	Closes the current list, dictionary, or function call.
10	(blank line)	Blank line. It separates blocks so the code is easier to read.
11	<code>for metric_name in repeated_split_metric_columns:</code>	Loops through each metric name and calculates its summary values.
12	<code>metric_values = best_qml_repeated_split_results_dataframe[metric_name]</code>	Selects one metric column from the repeated-split result table.
13	<code>mean_value = metric_values.mean()</code>	Calculates the average score over all repeated splits.
14	<code>standard_deviation_value = metric_values.std(ddof=1)</code>	Calculates how much the score changes between random splits.
15	<code>minimum_value = metric_values.min()</code>	Finds the lowest score seen in the repeated splits.
16	<code>maximum_value = metric_values.max()</code>	Finds the highest score seen in the repeated splits.
17	(blank line)	Blank line. It separates blocks so the code is easier to read.
18	<code>repeated_split_summary_rows.append(</code>	Starts an empty list for summary statistics across repeated train/test splits.
19	<code>{</code>	Starts a dictionary. In pandas table building, a dictionary gives column names and values for one row.
20	<code>"metric": metric_name,</code>	Sets the metric field in a dictionary row for a dataframe.
21	<code>"mean": mean_value,</code>	Sets the mean field in a dictionary row for a dataframe.
22	<code>"standard_deviation": standard_deviation_value,</code>	Sets the standard_deviation field in a dictionary row for a dataframe.
23	<code>"minimum": minimum_value,</code>	Sets the minimum field in a dictionary row for a dataframe.

24	"maximum": maximum_value,	Sets the maximum field in a dictionary row for a dataframe.
25	}	Closes the current list, dictionary, or function call.
26	)	Closes the current list, dictionary, dataframe, or function call.
27	(blank line)	Blank line. It separates blocks so the code is easier to read.
28	repeated_split_summary_dataframe = pd.DataFrame(repeated_split_summary_rows).round(4)	Rounds the number so the table is easier to read.
29	display(repeated_split_summary_dataframe)	Displays the dataframe output in the notebook.

Cell 27: Code

Purpose: Compares QML with Logistic Regression on the same repeated splits.

Important output: QML beats Logistic Regression on the mean repeated-split metrics in this saved experiment.

First line: qml_vs_logistic_summary_dataframe = read_project_data("qml vs logistic repeated split summary.csv")

Line	Code	Simple explanation
1	qml_vs_logistic_summary_dataframe = read_project_data("qml vs logistic repeated split summary.csv")	Loads the file qml vs logistic repeated split summary.csv into a dataframe.
2	(blank line)	Blank line. It separates blocks so the code is easier to read.
3	def get_mean_score(model_name, metric_name):	Defines a helper function to pull one mean score from the QML-vs-Logistic summary table.
4	model_rows = qml_vs_logistic_summary_dataframe[	Filters the summary table to one model.
5	qml_vs_logistic_summary_dataframe["model"] == model_name	Completes the current dataframe/list/scoring step in this cell.
6	]	Closes the current list, dictionary, or function call.
7	metric_rows = model_rows[model_rows["metric"] == metric_name]	Filters the model rows to one metric.
8	mean_score = metric_rows["mean"].iloc[0]	Takes the first row after sorting. In this notebook, that row is treated as the best result.
9	return mean_score	Completes the current dataframe/list/scoring step in this cell.
10	(blank line)	Blank line. It separates blocks so the code is easier to read.
11	qml_vs_logistic_comparison_rows = []	Completes the current dataframe/list/scoring step in this cell.
12	metric_names = ["accuracy", "stable_precision", "stable_recall", "stable_f1"]	Completes the current dataframe/list/scoring step in this cell.
13	(blank line)	Blank line. It separates blocks so the code is easier to read.
14	for metric_name in metric_names:	Loops through each metric name and calculates its summary values.
15	qml_mean = get_mean_score("QML kernel classifier", metric_name)	Completes the current dataframe/list/scoring step in this cell.

16	<code>logistic_mean = get_mean_score("Logistic Regression", metric_name)</code>	Completes the current dataframe/list/scoring step in this cell.
17	<code>difference = qml_mean - logistic_mean</code>	Calculates how much QML is ahead of or behind Logistic Regression.
18	(blank line)	Blank line. It separates blocks so the code is easier to read.
19	<code>if difference > 0:</code>	Checks whether QML scored higher for this metric.
20	<code>winner = "QML"</code>	Stores the model name that won for this metric.
21	<code>elif difference < 0:</code>	Checks whether Logistic Regression scored higher for this metric.
22	<code>winner = "Logistic Regression"</code>	Stores the model name that won for this metric.
23	<code>else:</code>	Handles the tie case.
24	<code>winner = "Tie"</code>	Stores the model name that won for this metric.
25	(blank line)	Blank line. It separates blocks so the code is easier to read.
26	<code>qml_vs_logistic_comparison_rows.append(</code>	Adds the current result row to a list so it can later become a dataframe.
27	<code>{</code>	Starts a dictionary. In pandas table building, a dictionary gives column names and values for one row.
28	<code>"metric": metric_name,</code>	Sets the metric field in a dictionary row for a dataframe.
29	<code>"qml_mean": round(qml_mean, 4),</code>	Rounds the number so the table is easier to read.
30	<code>"logistic_mean": round(logistic_mean, 4),</code>	Rounds the number so the table is easier to read.
31	<code>"qml_minus_logistic": round(difference, 4),</code>	Rounds the number so the table is easier to read.
32	<code>"winner": winner,</code>	Sets the winner field in a dictionary row for a dataframe.
33	<code>}</code>	Closes the current list, dictionary, or function call.
34	<code>)</code>	Closes the current list, dictionary, dataframe, or function call.
35	(blank line)	Blank line. It separates blocks so the code is easier to read.
36	<code>qml_vs_logistic_comparison_dataframe = pd.DataFrame(qml_vs_logistic_comparison_rows)</code>	Starts building a pandas dataframe so the notebook can show the values as a clean table.
37	<code>display(qml_vs_logistic_comparison_dataframe)</code>	Displays the dataframe output in the notebook.

Cell 28: Markdown

Purpose: Ends the presentation with the achievements, results, and safe interpretation.

First line: # Presentation Conclusion

Line	Notebook text	Meaning
1	# Presentation Conclusion	Main title of the notebook.
2	(blank line)	Blank line. It gives breathing space between text blocks.
3	**What we achieved**	Bold text used to highlight an important label or idea.

4	(blank line)	Blank line. It gives breathing space between text blocks.
5	- Built a complete lithium battery material pipeline.	Bullet point that summarizes one achievement or result.
6	- Created India-focused material scoring and final shortlist.	Bullet point that summarizes one achievement or result.
7	- Added QML-primary DSS recommendation rankings for exact compound formulas.	Bullet point that summarizes one achievement or result.
8	- Trained XGBoost as the strong present-day classical benchmark.	Bullet point that summarizes one achievement or result.
9	- Prepared a balanced QML-ready dataset.	Bullet point that summarizes one achievement or result.
10	- Trained a first Qiskit Statevector quantum-kernel classifier as the quantum-future	Bullet point that summarizes one achievement or result.
11	experiment.	Explanation text written for the presentation audience.
12	- Tuned QML hyperparameters using 4-fold cross-validation.	Bullet point that summarizes one achievement or result.
13	- Added exhaustive QML feature-combination tuning with 8,388 saved	Bullet point that summarizes one achievement or result.
14	configurations.	Explanation text written for the presentation audience.
15	- Added a separate improved-QML section using feature importance, PCA, and an	Bullet point that summarizes one achievement or result.
16	entangled-kernel search.	Explanation text written for the presentation audience.
17	- Added a threshold experiment for the improved-QML stable probability.	Bullet point that summarizes one achievement or result.
18	- Added a kernel-alignment experiment for quantum-aware feature selection.	Bullet point that summarizes one achievement or result.
19	- Validated the best QML setup across 10 random train/test splits.	Bullet point that summarizes one achievement or result.
20	- Compared best QML with Logistic Regression on the same repeated splits.	Bullet point that summarizes one achievement or result.
21	- Added a gate-level visual diagram for the best 4-qubit QML feature map.	Bullet point that summarizes one achievement or result.
22	(blank line)	Blank line. It gives breathing space between text blocks.
23	**Main model result**	Bold text used to highlight an important label or idea.
24	(blank line)	Blank line. It gives breathing space between text blocks.
25	- XGBoost full-project accuracy: **0.9091**	Bullet point that summarizes one achievement or result.

26	- XGBoost regressor MAE: **0.1005**	Bullet point that summarizes one achievement or result.
27	- QML accuracy on QML-ready test split: **0.8100**	Bullet point that summarizes one achievement or result.
28	- Tuned QML accuracy on QML-ready test split: **0.8200**	Bullet point that summarizes one achievement or result.
29	- Tuned QML stable F1 on QML-ready test split: **0.8269**	Bullet point that summarizes one achievement or result.
30	- Improved QML separate-section accuracy: **0.8200**	Bullet point that summarizes one achievement or result.
31	- Improved QML separate-section stable F1: **0.8235**	Bullet point that summarizes one achievement or result.
32	- Improved QML threshold-tuned accuracy: **0.8300**	Bullet point that summarizes one achievement or result.
33	- Improved QML threshold-tuned stable F1: **0.8381**	Bullet point that summarizes one achievement or result.
34	- Improved QML kernel-alignment accuracy: **0.8200**	Bullet point that summarizes one achievement or result.
35	- Improved QML kernel-alignment stable F1: **0.8302**	Bullet point that summarizes one achievement or result.
36	- Repeated-split mean accuracy: **0.8550**	Bullet point that summarizes one achievement or result.
37	- Repeated-split mean stable F1: **0.8583**	Bullet point that summarizes one achievement or result.
38	- QML vs Logistic mean accuracy: **0.8550 vs 0.8410**	Bullet point that summarizes one achievement or result.
39	- QML vs Logistic mean stable F1: **0.8583 vs 0.8473**	Bullet point that summarizes one achievement or result.
40	- Same-data XGBoost accuracy: **0.8300**	Bullet point that summarizes one achievement or result.
41	(blank line)	Blank line. It gives breathing space between text blocks.
42	**Safe interpretation**	Bold text used to highlight an important label or idea.
43	(blank line)	Blank line. It gives breathing space between text blocks.
44	XGBoost is stronger on the full tabular benchmark, but the final DSS is	Explanation text written for the presentation audience.
45	quantum-led. QML gives the first recommendation signal and XGBoost is used as a	Explanation text written for the presentation audience.
46	corrective backup when QML is uncertain or when the models disagree. The safe	Explanation text written for the presentation audience.
47	student-level point is that quantum feature spaces are future-facing for	Explanation text written for the presentation audience.

48	materials discovery, while classical ML can still act as a practical safety	Explanation text written for the presentation audience.
49	check today.	Explanation text written for the presentation audience.

9. Final Safe Explanation For Presentation

The project is best explained in this order:

1. We started with a large public Materials Project snapshot.
2. We filtered and scored lithium-containing materials for an India-focused decision problem.
3. We created a final DSS ranking of exact compound formulas.
4. We trained and compared classical ML and Qiskit-based QML results.
5. We used QML as the future-facing recommendation signal and XGBoost as a corrective support signal.
6. We do not claim real quantum hardware advantage. We claim a student-level QML simulator experiment inside a practical DSS pipeline.